

Zewail City of Science, Technology, and Innovation
University of Science and Technology
Communications and Information Engineering



AI-Driven Intelligent Load Management System for Optimized Resource Allocation in Distributed Networks

Graduation Project

Prepared By

Tasneem Muhammed Attia	202101031
Nada Nabil Soudi	202101220
Rghda Salah Ahmed	202101510

Supervised By

Dr. Tamer Ashour
Dr. Doaa Shawky

Contents

1	Introduction	3
2	System Overview	3
3	Revised System Design	4
3.1	Layered Architecture	4
3.1.1	Ingress and Traffic Routing Layer	4
3.1.2	Telemetry and Observability Layer	5
3.1.3	Decision Intelligence Layer	5
3.1.4	Control and Actuation Layer	5
3.2	Communication Channels	6
4	Data Flow	7
5	Control Loop Architecture	7
6	Request Processing Sequence	9
7	Deployment Architecture	9
8	Design Constraints	10
8.1	Constraint Summary	11
8.2	Scalability	11
8.3	Extensibility	11
8.4	Maintainability	11
8.5	Interoperability	11
8.6	Reliability	11
9	Initial Implementation	12
9.1	Technologies Used	12

9.2	Implementation Architecture	12
9.3	Load Balancing Infrastructure	13
9.4	Health Monitoring	13
9.5	Telemetry and Logging	13
9.6	Containerized Deployment	14
10	Testing and Evaluation	14
11	Standards Used	14

1 Introduction

Modern distributed systems must handle dynamic workloads while maintaining high availability and performance. Traditional load balancers rely on static routing strategies such as round robin or least connections, which may not adapt well to changing traffic patterns or infrastructure conditions.

SmartLoad is proposed as an intelligent middleware platform that enhances traditional load balancing with telemetry-driven decision making and machine learning based optimization.

The system continuously collects telemetry metrics, analyzes them using decision intelligence models, and dynamically adjusts routing policies and infrastructure scaling.

2 System Overview

SmartLoad operates as a middleware platform between client applications and backend service pools.

Its main responsibilities include:

- Intelligent request routing
- Real-time telemetry collection
- Predictive workload analysis
- Adaptive infrastructure scaling

Figure 1 shows the high-level system context.

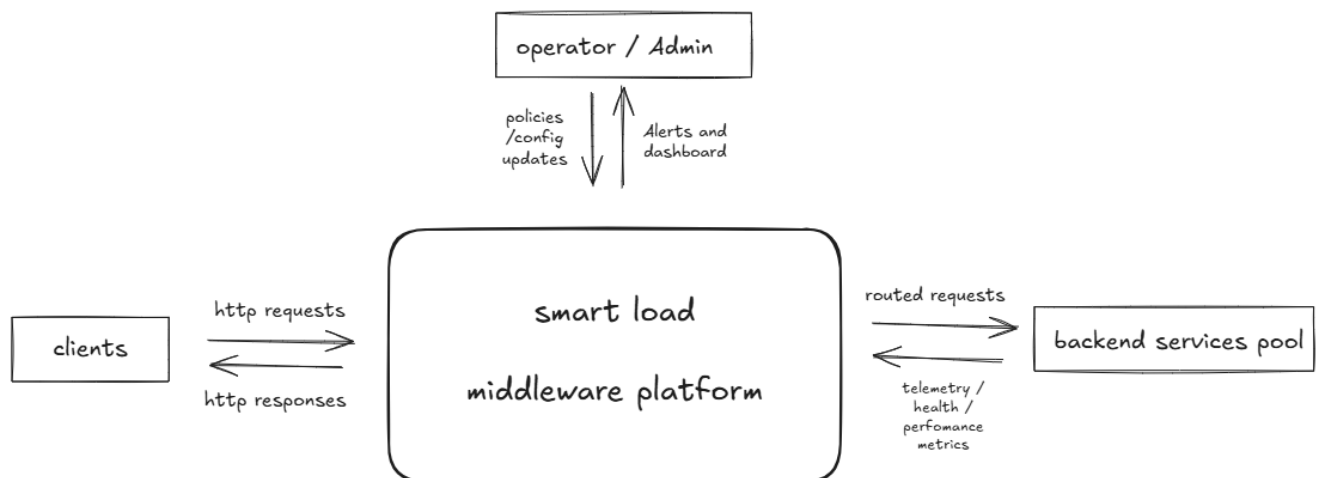


Figure 1: SmartLoad System Context

Clients send HTTP requests to the SmartLoad platform, which routes them to backend services while continuously monitoring system performance and adapting routing policies.

3 Revised System Design

The SmartLoad architecture is organized into multiple logical layers that separate system responsibilities and improve maintainability.

3.1 Layered Architecture

The platform is divided into four major layers:

- Ingress and Traffic Routing Layer
- Telemetry and Observability Layer
- Decision Intelligence Layer
- Control and Actuation Layer

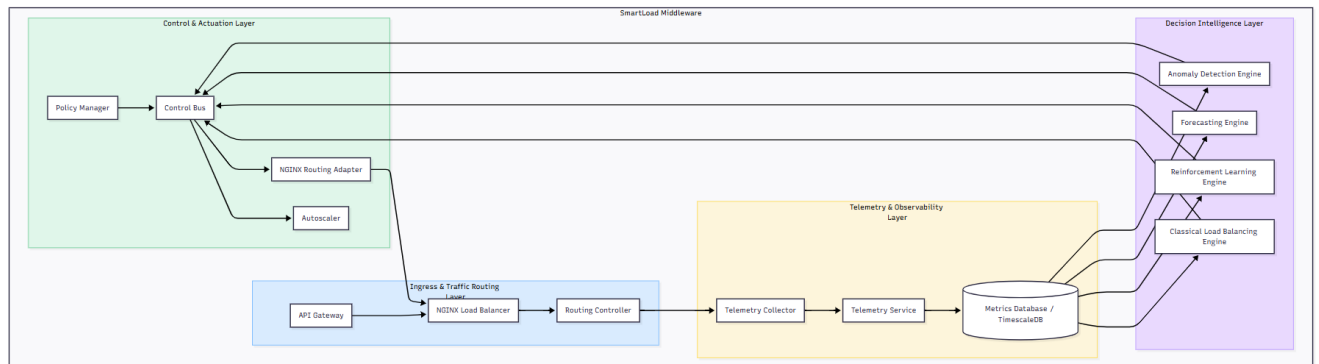


Figure 2: Layered Architecture of SmartLoad

3.1.1 Ingress and Traffic Routing Layer

This layer handles incoming requests and forwards them to backend services.

Key components include:

- API Gateway
- NGINX Load Balancer
- Routing Controller

The load balancer distributes incoming traffic while the routing controller dynamically updates routing policies based on system intelligence.

3.1.2 Telemetry and Observability Layer

This layer collects operational metrics from the system and stores them for analysis.

Components include:

- Telemetry Collector
- Telemetry Service
- Metrics Database (TimescaleDB)

Telemetry data includes latency, request counts, routing decisions, and backend performance metrics.

3.1.3 Decision Intelligence Layer

This layer analyzes telemetry metrics to generate routing and scaling decisions.

Modules include:

- Forecasting Engine
- Reinforcement Learning Engine
- Anomaly Detection Engine
- Classical Load Balancing Engine

These models produce routing scores, anomaly alerts, and workload forecasts.

3.1.4 Control and Actuation Layer

The control layer applies system decisions by updating routing rules and scaling infrastructure.

Components include:

- Control Bus (Redis Pub/Sub)
- NGINX Routing Controller
- Autoscaler
- Policy Manager

3.2 Communication Channels

SmartLoad uses multiple communication technologies between system components.

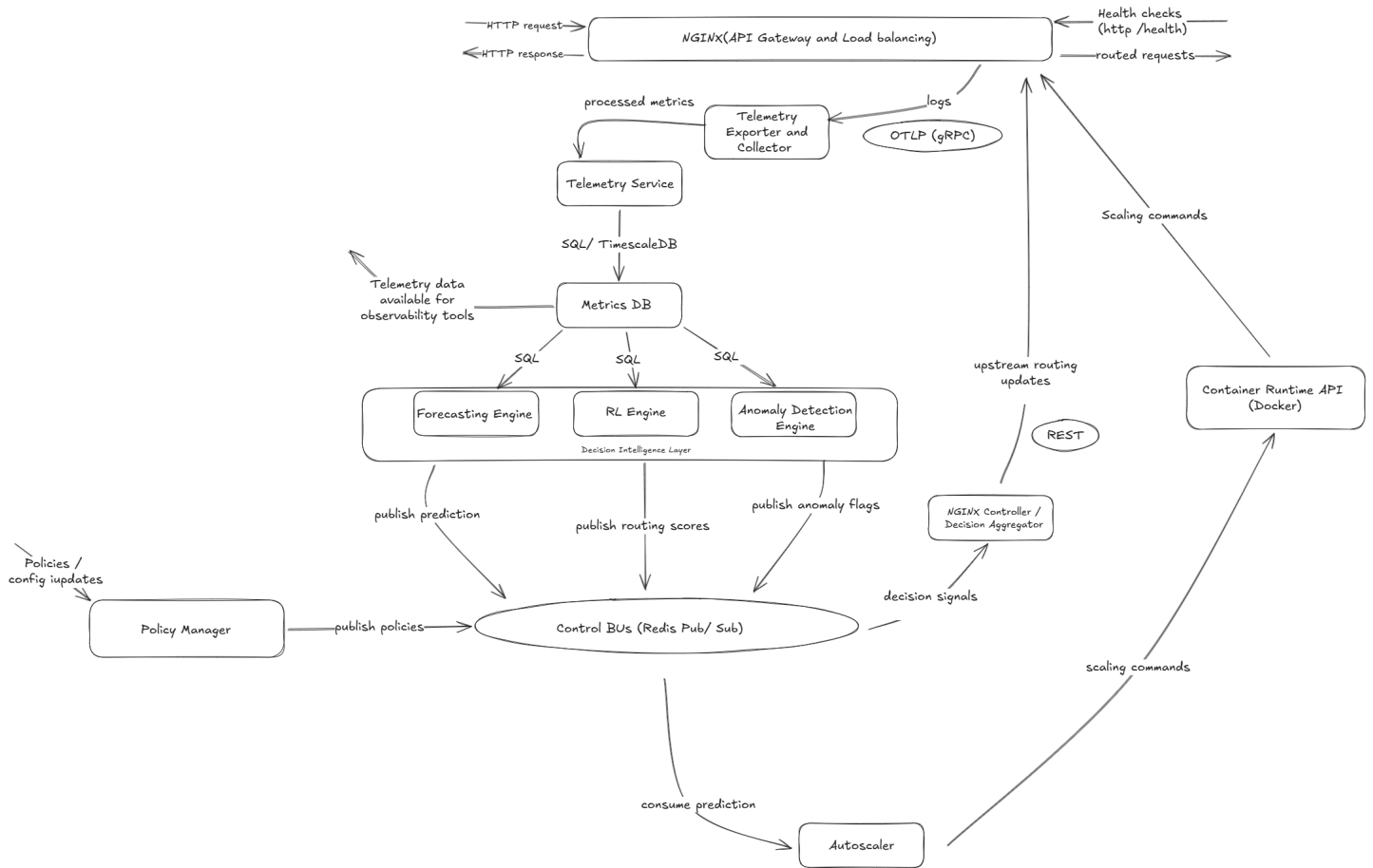


Figure 3: Communication Channels Between SmartLoad Services

The following communication mechanisms are used:

- **HTTP / REST**

Used for request routing and control operations.

- **Redis Pub/Sub**

Used as the Control Bus for asynchronous communication between decision engines and controllers.

- **OTLP (gRPC)**

Used by OpenTelemetry to transmit telemetry logs and metrics.

- **SQL**

Used to store and retrieve telemetry metrics from the metrics database.

These technologies allow the system to support both real-time control signals and persistent data storage.

4 Data Flow

The SmartLoad platform implements a telemetry-driven data pipeline.

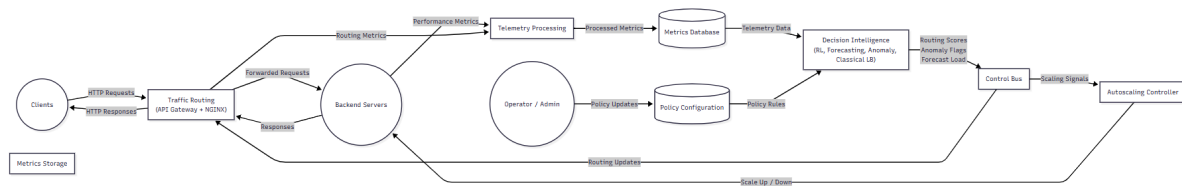


Figure 4: SmartLoad Data Flow Architecture

The data flow process consists of the following steps:

1. Client requests enter the system through the API gateway.
2. Requests are routed by the NGINX load balancer.
3. Telemetry data is generated during request processing.
4. Metrics are processed and stored in the metrics database.
5. Decision intelligence modules analyze system performance.
6. Routing and scaling decisions are published to the control bus.

5 Control Loop Architecture

SmartLoad operates using a closed feedback loop where system monitoring drives adaptive control actions.

The control loop performs the following cycle:

1. Monitor system telemetry
2. Analyze metrics using decision intelligence models
3. Generate routing and scaling decisions
4. Apply decisions through system controllers

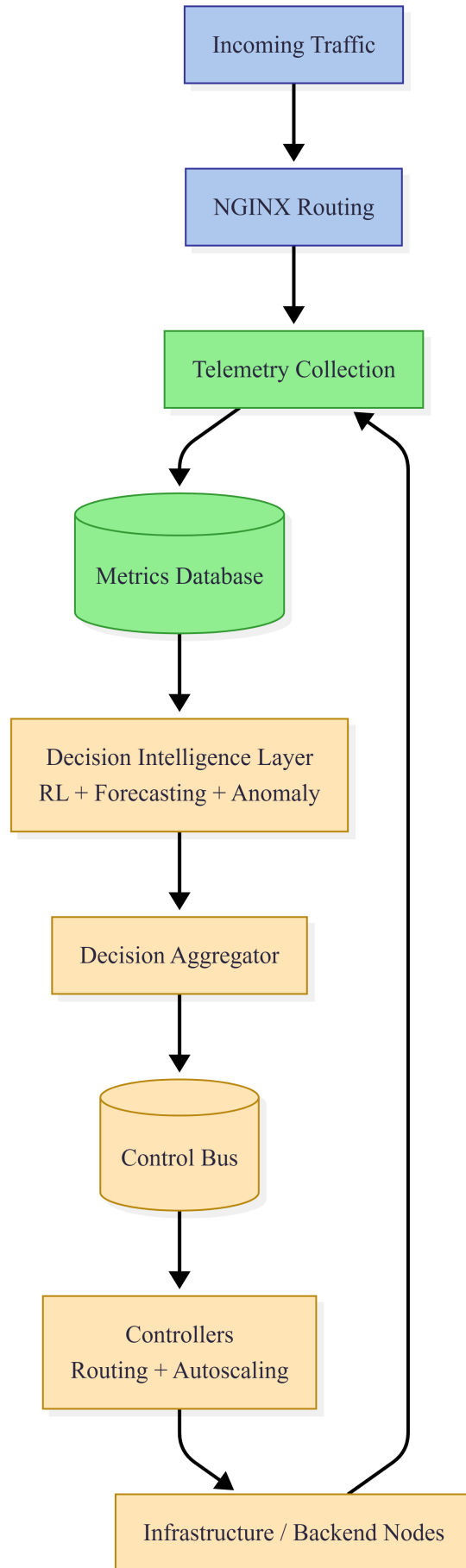


Figure 5: SmartLoad Feedback Control Loop

6 Request Processing Sequence

Figure 6 illustrates how a client request flows through the SmartLoad platform.

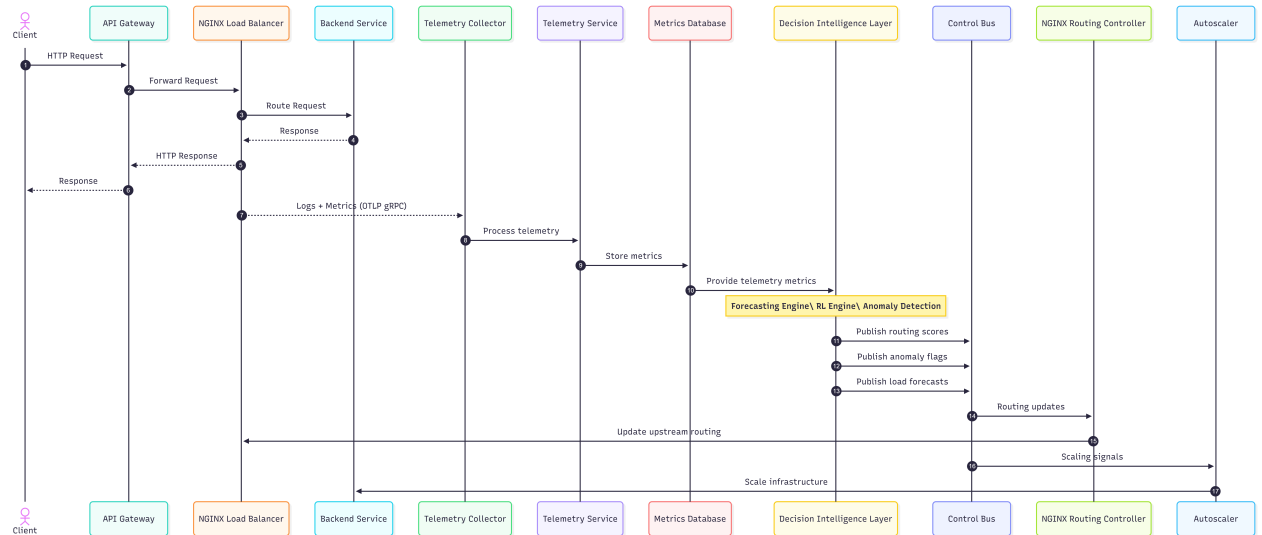


Figure 6: Request Processing Sequence

The sequence includes request routing, telemetry collection, decision generation, and infrastructure updates.

7 Deployment Architecture

SmartLoad is implemented using containerized microservices.

System components are deployed across multiple infrastructure nodes including ingress nodes, telemetry nodes, decision intelligence nodes, and control nodes.

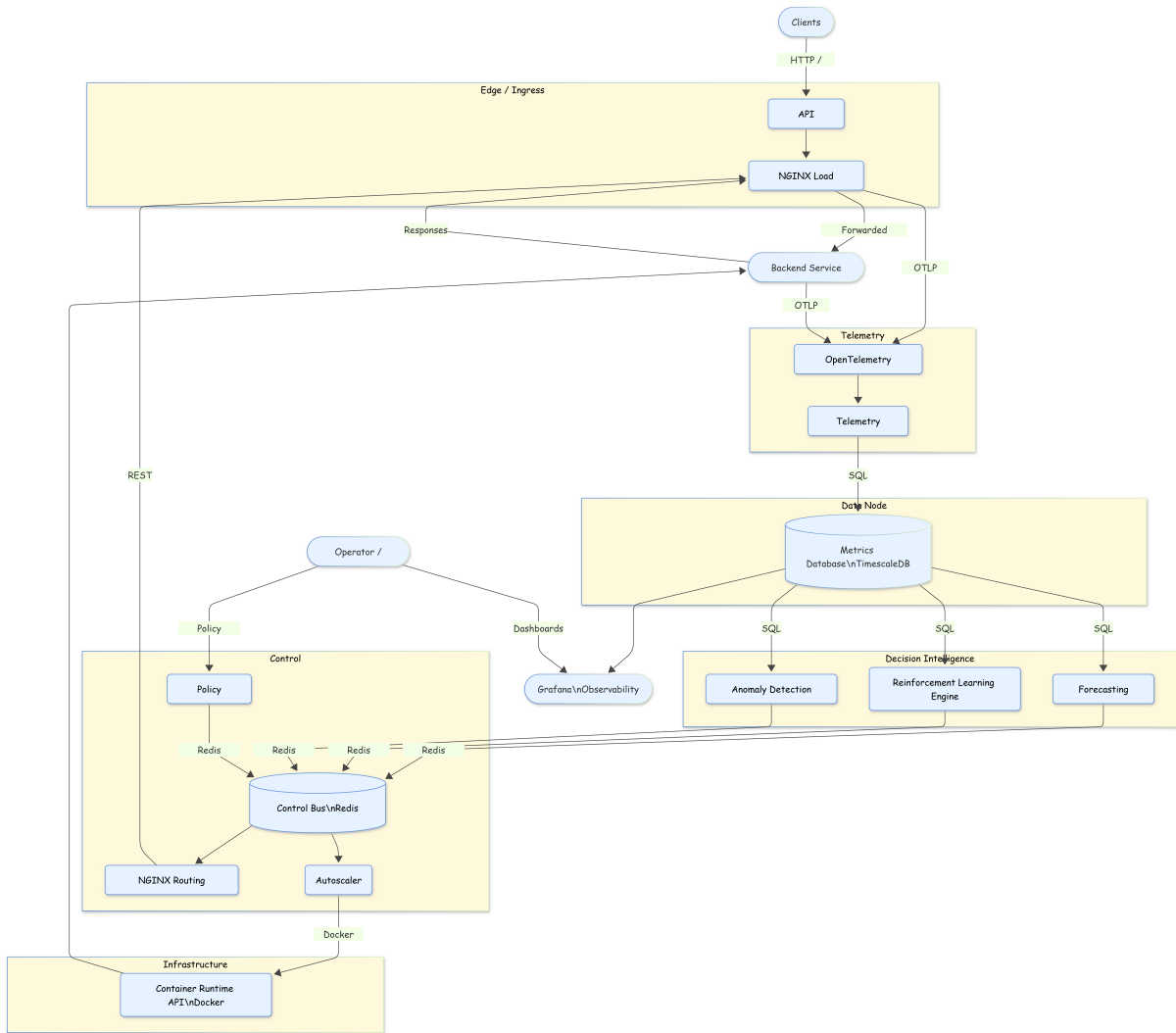


Figure 7: SmartLoad Deployment Architecture

System components are deployed across multiple infrastructure nodes including ingress nodes, telemetry nodes, decision intelligence nodes, and control nodes.

8 Design Constraints

The SmartLoad architecture is designed under several engineering constraints that guide system scalability, performance, and integration with existing infrastructure.

8.1 Constraint Summary

Constraint	Target Value	Description
Routing Latency Overhead	< 5 ms	Additional delay introduced by SmartLoad ingress
System Throughput	500–1000 req/s	Request capacity in prototype environment
Telemetry Update Interval	1–2 s	Time for telemetry to influence routing decisions
Failure Detection Time	< 10 s	Time to detect and isolate unhealthy backend nodes
Autoscaling Response	< 30 s	Time required to provision additional backend capacity

Table 1: SmartLoad Design Targets

8.2 Scalability

SmartLoad must support increasing request loads without degrading system performance. Horizontal scalability is achieved by dynamically adding backend service instances through the autoscaling component.

8.3 Extensibility

The architecture separates routing, telemetry, and decision intelligence services. This modular structure allows new models or decision components to be integrated without major system modifications.

8.4 Maintainability

SmartLoad follows a microservice architecture where each subsystem operates independently. This simplifies debugging, testing, and system updates.

8.5 Interoperability

The system integrates with common cloud-native infrastructure technologies such as Docker containers, HTTP-based services, and NGINX load balancing.

8.6 Reliability

Backend health monitoring ensures that failing nodes are detected quickly and removed from the routing pool to prevent degraded service.

9 Initial Implementation

The initial SmartLoad prototype focuses on implementing the core routing infrastructure and basic monitoring mechanisms. The prototype includes an NGINX-based load balancer, multiple backend service instances, a health monitoring routine, and containerized deployment using Docker.

9.1 Technologies Used

Component	Technology
Load Balancer	NGINX
Backend Services	Node.js HTTP servers
Health Monitoring	Custom health-check routine
Telemetry / Logging	NGINX access logs
Workload Simulation	Locust
Container Platform	Docker

Table 2: Technologies used in the SmartLoad initial prototype

9.2 Implementation Architecture

Figure ?? shows the architecture of the initial SmartLoad prototype. Client applications send HTTP requests to the NGINX load balancer, which distributes incoming traffic across multiple backend service instances. Backend services run as independent containers and process requests forwarded by the load balancer.

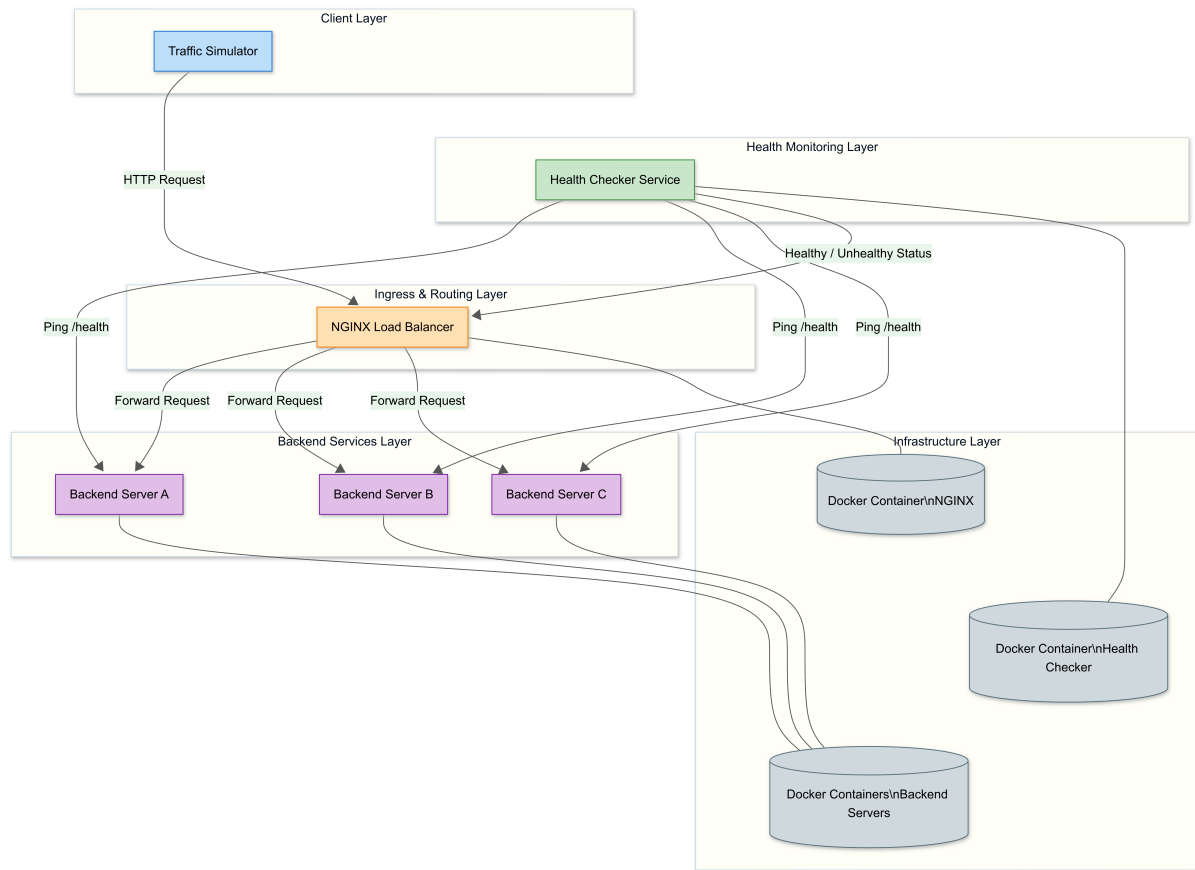


Figure 8: Interaction between load balancer, health checker, and backend servers

9.3 Load Balancing Infrastructure

The ingress routing layer is implemented using NGINX. Incoming HTTP requests are received by the load balancer and forwarded to available backend instances. This component acts as the central routing point of the system and enables traffic distribution across multiple services.

9.4 Health Monitoring

A health-check routine periodically sends `/health` requests to backend servers to verify their availability. If a backend instance fails to respond or returns an error, it is temporarily removed from the routing pool until it recovers.

9.5 Telemetry and Logging

The load balancer generates structured access logs for each incoming request. These logs include the request path, response status, client address, and the selected backend server. Logging provides the first layer of telemetry that will later feed the SmartLoad monitoring and decision components.

9.6 Containerized Deployment

All system components are deployed as Docker containers. The NGINX load balancer, backend services, and monitoring components run in separate containers and communicate over the container network. This setup allows the prototype to simulate a distributed environment while remaining easy to deploy and reproduce.

10 Testing and Evaluation

Testing was conducted using simulated backend servers to evaluate routing behavior and system monitoring.

The evaluation focused on:

- Request routing correctness
- Backend failure detection
- Telemetry accuracy
- Autoscaling responsiveness

Several test scenarios were performed including backend failure simulation and traffic burst testing.

The system successfully detected backend failures and adjusted routing behavior accordingly.

11 Standards Used

The SmartLoad implementation follows several industry standards.

- HTTP/REST communication protocols
- OpenTelemetry observability standard
- Redis Pub/Sub event communication
- SQL database standards
- Docker containerization

These standards ensure interoperability and future extensibility.